STEP 1 — Install Libraries

```
!pip install -q torch transformers datasets peft accelerate
```

STEP 2 — Import Libraries

```
import torch
from transformers import AutoModelForSequenceClassification, AutoTokenizer
from datasets import load_dataset
from peft import LoraConfig, get_peft_model
```

STEP 3 — Load Pre-trained Model

```
model_name = "distilbert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2
)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
config.json: 100%                                                 483/483 [00:00<00:00, 54.0kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
tokenizer_config.json: 100%                                       48.0/48.0 [00:00<00:00, 2.46kB/s]
vocab.txt: 232k/? [00:00<00:00, 3.11MB/s]
tokenizer.json: 466k/? [00:00<00:00, 6.59MB/s]
model.safetensors: 100%                                           268M/268M [00:02<00:00, 253MB/s]
Loading weights: 100%                                 100/100 [00:00<00:00, 197.86it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.weight]
DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key                       | Status     |
--------------------------+------------+-
vocab_transform.weight    | UNEXPECTED |
vocab_layer_norm.bias     | UNEXPECTED |
vocab_layer_norm.weight   | UNEXPECTED |
vocab_projector.bias      | UNEXPECTED |
vocab_transform.bias      | UNEXPECTED |
pre_classifier.bias       | MISSING    |
classifier.bias           | MISSING    |
classifier.weight         | MISSING    |
pre_classifier.weight     | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
```

STEP 4 — Load Dataset

```
dataset = load_dataset("imdb")
```

```
README.md:          7.81k/? [00:00<00:00, 161kB/s]
plain_text/train-00000-of-00001.parquet: 100%  21.0M/21.0M [00:00<00:00, 105MB/s]
plain_text/test-00000-of-00001.parquet: 100%  20.5M/20.5M [00:00<00:00, 101MB/s]
plain_text/unsupervised-00000-of-00001.p(…): 100%  42.0M/42.0M [00:00<00:00, 109MB/s]
Generating train split: 100%  25000/25000 [00:00<00:00, 47269.04 examples/s]
Generating test split: 100%  25000/25000 [00:00<00:00, 61580.00 examples/s]
Generating unsupervised split: 100%  50000/50000 [00:00<00:00, 86085.83 examples/s]
```

STEP 5 — Tokenization

```
model_name = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)

def tokenize_function(example):
    return tokenizer(example["text"], truncation=True, padding="max_length")

tokenized_dataset = dataset.map(tokenize_function, batched=True)
```

```
Map: 100%  25000/25000 [00:46<00:00, 919.51 examples/s]
Map: 100%  25000/25000 [00:25<00:00, 1078.36 examples/s]
Map: 100%  50000/50000 [00:51<00:00, 982.97 examples/s]
```

STEP 6 — Reduce Dataset Size (Faster Training)

```
train_dataset = tokenized_dataset["train"].shuffle(seed=42).select(range(500))
test_dataset = tokenized_dataset["test"].shuffle(seed=42).select(range(200))
```

STEP 7 — Configure LoRA

```
lora_config = LoraConfig(
    r=8,
    lora_alpha=16,
    target_modules=["q_lin", "v_lin"],  # for DistilBERT
    lora_dropout=0.1,
    bias="none",
    task_type="SEQ_CLS"
)
```

STEP 8 — Apply PEFT Model

```
peft_model = get_peft_model(model, lora_config)

peft_model.print_trainable_parameters()
```

```
trainable params: 739,586 || all params: 67,694,596 || trainable%: 1.0925
```

STEP 9 — Training Setup

```
from transformers import TrainingArguments, Trainer

training_args = TrainingArguments(
    output_dir="./results",
    learning_rate=2e-4,
    per_device_train_batch_size=8,
    num_train_epochs=1,
    logging_steps=10,
    save_strategy="no"
)
```

STEP 10 — Train Model

```
trainer = Trainer(
    model=peft_model,
    args=training_args,
    train_dataset=train_dataset
)

trainer.train()
```

[63/63 00:16, Epoch 1/1]

Step	Training Loss
10	0.702583
20	0.668578
30	0.666298
40	0.637819
50	0.639344
60	0.623978

```
TrainOutput(global_step=63, training_loss=0.6544376933385455, metrics={'train_runtime': 17.8178, 'train_samples_per_second':
28.062, 'train_steps_per_second': 3.536, 'total_flos': 67369703424000.0, 'train_loss': 0.6544376933385455, 'epoch': 1.0})
```

STEP 11 — Evaluate Model

```
test_dataset = tokenized_dataset["test"].select(range(50))
```

STEP 12 — Test Prediction

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

peft_model.to(device)   # move model to device

text = "The movie was fantastic and inspiring!"

inputs = tokenizer(text, return_tensors="pt").to(device)  # move inputs to same device

outputs = peft_model(**inputs)

prediction = torch.argmax(outputs.logits, dim=1)

print("Predicted label:", prediction.item())
```

```
Predicted label: 1
```

```
Start coding or generate with AI.
```